

Phase 04 — Linux Server Administration (Complete Notes)

Production servers are Linux. This phase makes you comfortable operating one.

1. Why Linux?

~96% of the top web servers run Linux: free, open source, stable for years without reboot, scriptable, secure multi-user design, runs on everything. **Ubuntu Server LTS** (Long Term Support, 5 years of updates — e.g. 24.04) is the common choice and what we'll use. A server edition has no GUI — everything is the terminal. That's a feature: fewer resources, fewer attack surfaces.

Analogy: GUI = automatic car; terminal = manual transmission. Production drivers drive manual.

2. The filesystem

One tree, starting at root `/` (no `C:\` drives):

```
/
├─ home/raju/    your files (~ = your home)
├─ root/         root user's home
├─ etc/          ★ CONFIGURATION (nginx, ssh, systemd units...)
├─ var/          variable data: /var/log ★ LOGS, /var/www sites, /var/lib/postgresql data
├─ usr/          installed programs (/usr/bin, /usr/local)
├─ bin/, sbin/   essential commands
├─ opt/          optional/self-installed software (your app can live here)
├─ tmp/          temp files, cleared on reboot
├─ proc/, sys/   virtual: kernel & process info as "files"
└─ dev/          devices as files
```

For ops work you live in three places: **/etc** (config), **/var/log** (what went wrong), and your app's directory.

Essential navigation & file commands

```
pwd; ls -lah; cd /var/log      # where am I, what's here, go
cat file; less file           # view (less: q quits, / searches)
tail -f /var/log/syslog       # ★ follow a log live – your daily tool
head -20 file
cp src dst; mv src dst; rm file; rm -rf dir    # careful with rm -rf!
mkdir -p a/b/c; touch file
find /etc -name "*.conf"
grep -r "error" /var/log/ 2>/dev/null          # search text in files
df -h                                           # disk space (full disk = classic outage)
du -sh /var/log/*                             # what's eating the disk
```



```

ps aux                                # all processes (USER PID %CPU %MEM COMMAND)
ps aux | grep java
top      # live view – press M (sort by memory), P (cpu), q
htop      # nicer (sudo apt install htop)
kill <PID>      # polite stop (SIGTERM – app can clean up)
kill -9 <PID>    # force kill (SIGKILL – last resort, no cleanup)
java -jar app.jar &  # & = background; jobs, fg bring it back
nohup java -jar app.jar &  # survives logout – but the real answer is systemd ↓

```

6. systemd — how services run in production

You never keep an app alive with `nohup`. **systemd** is Ubuntu's init system: starts services at boot, restarts them on crash, manages dependencies and logs.

A real unit file for Spring Boot — memorize this shape

`/etc/systemd/system/myapp.service` :

```

[Unit]
Description=My Spring Boot API
After=network.target postgresql.service

[Service]
User=appuser
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/java -jar /opt/myapp/app.jar
Restart=always
RestartSec=5
Environment=SPRING_PROFILES_ACTIVE=prod

[Install]
WantedBy=multi-user.target

```

```

sudo systemctl daemon-reload      # re-read unit files after editing
sudo systemctl start myapp
sudo systemctl status myapp        # ★ first command when anything is wrong
sudo systemctl enable myapp        # start automatically at boot
sudo systemctl restart myapp
systemctl list-units --type=service --state=running

```

`Restart=always` = crashed app auto-restarts in 5s. This single line is your first taste of self-healing infrastructure.

7. Logs & journalctl

Two worlds:

- **journald** (systemd's log db): everything services print → `journalctl`
- Classic files in `/var/log/` : `syslog` , `auth.log` (★ login attempts — watch bots brute-force you), `nginx/access.log` , `nginx/error.log`

```
journalctl -u myapp           # logs of one service
journalctl -u myapp -f        # * follow live (like tail -f)
journalctl -u myapp --since "10 min ago"
journalctl -u myapp -p err    # errors only
sudo tail -f /var/log/nginx/access.log
sudo grep "Failed password" /var/log/auth.log | tail
```

Debugging ritual when a service is down: `systemctl status X` → `journalctl -u X -n 50` → fix → `restart` → `status` .

8. Firewall — ufw

Default stance: **deny everything inbound, allow only what you serve.**

```
sudo ufw allow OpenSSH        # * FIRST – or you lock yourself out of the server
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose
```

Note what's NOT allowed: 8080 (Spring Boot — only NGINX reaches it, locally), 5432 (PostgreSQL — never expose your DB to the Internet; the #1 rookie breach).

9. Networking commands on a server

```
ip addr; ip route             # my IPs / my gateway
ss -tulpn                     # * what is listening on which port
ping -c3 8.8.8.8              # do I have Internet (L3)?
dig example.com +short        # does DNS work?
curl -I localhost:8080        # does my app answer locally (L7)?
nc -zv db-host 5432           # is a remote port reachable (L4)?
```

This is the Phase-1 debugging ladder, now with tools: **L3 ping** → **L4 nc/ss** → **L7 curl**.

10. Common mistakes

- Working as root for everything / running apps as root.
- `ufw enable` before allowing SSH → locked out of your own server.
- `rm -rf` with a mistyped path; no backups.
- Killing with `-9` first (skips graceful shutdown, can corrupt state).
- Ignoring disk: logs fill the disk → DB crashes. `df -h` weekly; configure logrotate.
- Editing config without a backup copy and without testing (`nginx -t` exists for a reason — Phase 5).
- `chmod 777` "to make it work" — you've made it work for attackers too.

11. Best practices

- Dedicated user per app; SSH keys only; ufw default-deny.
- Everything long-running is a systemd unit with `Restart=always` .
- Know your logs before the incident, not during.
- `apt update && apt upgrade` regularly; unattended-upgrades for security patches.

12. Interview questions

1. What does `chmod 750` mean exactly? Why are SSH private keys `600`?
2. Why run applications as a non-root user?
3. How do SSH keys work? (Tie it to public/private crypto from Phase 3.)
4. Your service died at 3am. Walk through diagnosing it (`status` → `journalctl` → `fix` → `restart`).
5. `kill` vs `kill -9`?
6. What does `systemd` give you over `nohup java -jar &`?
7. Server disk is 100% full — how do you find the culprit and recover?
8. Which ports should be open on a typical web server, and why is 5432 not one of them?

13. LAB — build a mini server locally

Best done in a VM (VirtualBox/Multipass: `multipass launch --name lab`) or any Ubuntu machine:

```
# 1. create the app user and a fake app
sudo adduser appuser
echo -e '#!/bin/bash\nwhile true; do echo "app alive $(date)"; sleep 5; done' | sudo tee
/opt/fakeapp.sh
sudo chmod 755 /opt/fakeapp.sh && sudo chown appuser /opt/fakeapp.sh

# 2. make it a service (unit file as in §6, ExecStart=/opt/fakeapp.sh, User=appuser)
sudo systemctl daemon-reload && sudo systemctl start fakeapp && systemctl status fakeapp

# 3. watch logs live, then kill the process — watch systemd resurrect it
journalctl -u fakeapp -f          # terminal 1
ps aux | grep fakeapp; sudo kill -9 <PID>; ps aux | grep fakeapp  # terminal 2 — new PID!

# 4. firewall
sudo ufw allow OpenSSH && sudo ufw enable && sudo ufw status
```

14. ASSIGNMENT 04 (submit to Claude)

1. Do the lab; paste `systemctl status fakeapp` output showing an auto-restart (look at the PID change / uptime).
2. Write, from memory, a `systemd` unit file for a Spring Boot app `shop-api.jar` in `/opt/shop`, running as `shopuser`, prod profile, auto-restart, starting after PostgreSQL.
3. Explain each field of this line: `-rw-r----- 1 shopuser www-data 2048 Jul 19 10:00 application.yml` — and give the `chmod` number.
4. Scenario: `curl https://myapi.com` times out. Give your full diagnosis sequence, from DNS to app, naming the command for each step and what each result would tell you.
5. List 5 things you'd do to a brand-new Ubuntu VPS in the first 30 minutes, in order, with commands.